This may sound well and good, but one potential problem is that queries that were written against the old index (`GET cars/_search { .. }`, for example) need to be updated because they now need to run against a new index (`cars_new`). If these queries are hardcoded in the application code, we may need a hotfix release to production.

Say we have an index called `vintage_cars` with data about vintage cars, and we are required to update the index. Here's where we can lean on aliasing; we can devise a strategy with an alias in mind. By executing the following steps (also illustrated in figure 6.4), we can probably achieve the migration with zero downtime:

1. Create an alias called `vintage_cars_alias` to refer to the current index `vintage_cars`.
2. Because the new properties are incompatible with the existing index, create a new index, `vintage_cars_new`, with the new settings.
3. Copy (i.e., reindex) the data from the old index (`vintage_cars`) to the new index (`vintage_cars_new`).
4. Re-create the existing alias (`vintage_cars_alias`), which pointed to the old index, to refer to the new index. (See the next section.) Thus, `vintage_cars_alias` now points to `vintage_cars_new`.
5. All queries that were executed against the `vintage_cars_alias` are now carried out on the new index.
6. Delete the old index (`vintage_cars`) when reindexing and releasing have been shown to work.

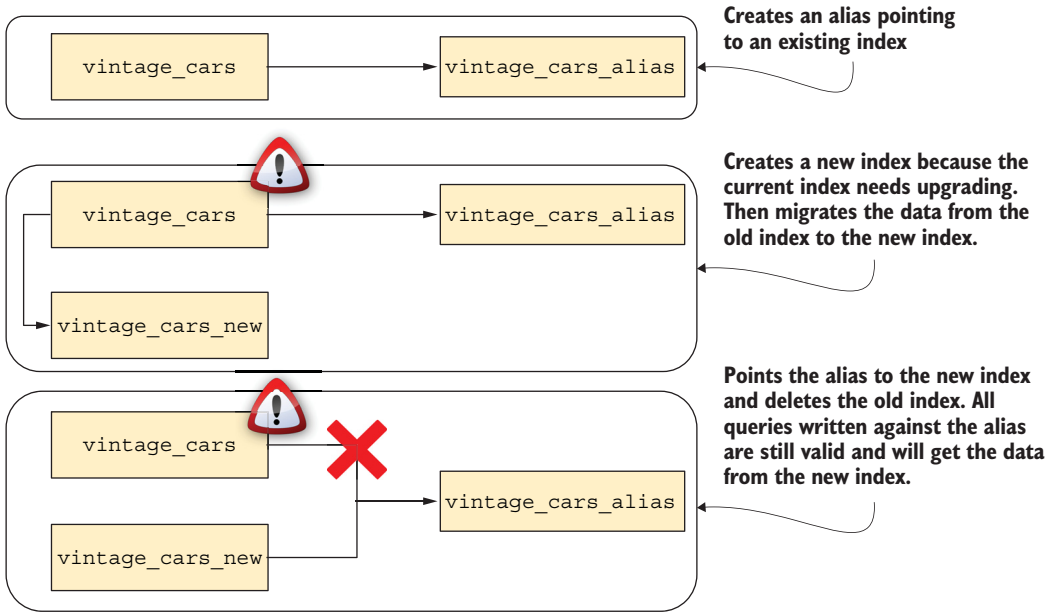


Figure 6.4 Achieving migration with zero downtime